```
    int tau^i d(tau^j)
    h = prob["T"] / n; pieces = []; C = np.zeros((q + 1, M)); C[0] = y
    for k in range(n):
        for _ in range(40):
            FJ = [rd.FJ(Pg[g] @ C, L, src) for g in range(len(tg))]
            Rr = Mt @ C - h * np.einsum("g,gi,gm->im", wg, Pg, np.array([f for f, _ in FJ])); Rr[0] += C[0] - y
            Jb = np.zeros(((q + 1) * M, (q + 1) * M))
            for i in range(q + 1):
                for j in range(q + 1):
                    blk = Mt[i, j] * np.eye(M) - h * sum(wg[g] * Pg[g, i] * Pg[g, j] * FJ[g][1] for g in range(len(tg)))
                    Jb[i * M:(i + 1) * M, j * M:(j + 1) * M] = blk + (np.eye(M) if i == j == 0 else 0)
            d = solve(Jb, Rr.ravel()).reshape(q + 1, M); C -= d
            if np.max(np.abs(d)) < newton_tol * max(1.0, np.max(np.abs(C))): break
        cs = C / (h ** np.arange(q + 1))[:, None]
        pieces.append(Piece(k * h, h, x, [cs[:, a * N:(a + 1) * N].T for a in range(m)]))
        y = C.sum(0); C = np.zeros_like(C); C[0] = y
    return pieces, [(0.0, x)]
```

pde_judge_nd.py — 경로 B의 임의 차원: 텐서곱 재구성, (d+1)차원 번스타인, 합집합 칸 도약, 국소화

```
"""pde_judge_nd: path B in any space dimension d >= 1.  A posteriori L1 bound against the CONTINUOUS solution of
    u_t = D_a (Laplacian u_a) + R_a(u),   a = 1..m,   x in the box [0, Lx]^d,   zero flux on every face,
  R polynomial.

Candidate: a sequence of pieces PieceND(t0, h, xs, V); xs is a list of d node arrays (a tensor mesh, which may change from
piece to piece), V[a] has shape (N_1, ..., N_d, K): the nodal time coefficients of species a in the local time s.  The
candidate may come from any order of discretization (space first, time first with remeshing, space-time); the judge does not ask.

Reconstruction: the tensor product of clamped C2 cubic splines, every axis with end slope 0.  Spline interpolation is linear,
so the tensor product is applied one axis at a time; the end slopes are exactly 0, hence the normal derivative of the
reconstruction is exactly 0 on every face.  On each cell x step the candidate is a polynomial in (xi_1..xi_d, s).

Residual r_a = u~_a,t - D_a Laplacian u~_a - R_a(u~): a polynomial on each cell x step; its L1 norm is bounded by the mean
absolute (d+1)-dimensional Bernstein coefficient times the cell volume times h, plus the rounding of forming it.

Theorem (same as pde_judge, any d).  Assume (H) the exact solution stays in a box B.  With mu_k an upper bound of the L1
log-norm of the reaction Jacobian on the hull of B and the candidate's range on piece k, the weak form with test function
sgn(e_a) and Kato's inequality  int sgn(e) Laplacian e <= int_boundary sgn(e) d_n e = 0  give
    d/dt ||e||_1 <= mu_k ||e||_1 + ||r||_1,   E_{k+1} = g_k (E_k + Gamma_k + iint_k |r|),   g_k = exp(max(mu_k, 0) h_k),
Gamma_k = ||u~(t_k+) - u~(t_k-)||_1 (jump between pieces, union-of-knots cells when the meshes differ).

localize(res): the same exact additive split as pde_judge.localize, cells now being boxes."""
import math, time
import numpy as np
from pde_judge import bern_matrix, spline_slopes, cubic_coeffs, Judge as _Judge1
from rig import gam
from core import P as _P
from math import comb

PADF = 1.0 + 1e-10


class QN:
    """polynomials in (xi_1, ..., xi_d, s) on a batch of cells: coefficient array (cells, p_1, ..., p_d, p_t)"""
    def __init__(s, c): s.c = np.asarray(c, float)
    @staticmethod
    def k(o, like):
        if isinstance(o, QN): return o
        c = np.zeros((like.c.shape[0],) + (1,) * (like.c.ndim - 1)); c.reshape(c.shape[0])[:] = o; return QN(c)
    def _pad(s, sh):
        Z = np.zeros((s.c.shape[0],) + tuple(sh)); Z[(slice(None),) + tuple(slice(0, n) for n in s.c.sha
```

```
    pe[1:])] = s.c; return Z
41      def __add__(s, o):
42          o = QN.k(o, s); sh = [max(a, b) for a, b in zip(s.c.shape[1:], o.c.shape[1:])]; return QN(s._pad
    (sh) + o._pad(sh))
43      __radd__ = __add__
44      def __neg__(s): return QN(-s.c)
45      def __sub__(s, o): return s + (-QN.k(o, s))
46      def __rsub__(s, o): return QN.k(o, s) + (-s)
47      def __mul__(s, o):
48          if not isinstance(o, QN): return QN(s.c * float(o))
49          a, b = (s, o) if np.prod(s.c.shape[1:]) <= np.prod(o.c.shape[1:]) else (o, s)
50          sa, sb = a.c.shape[1:], b.c.shape[1:]
51          C = np.zeros((a.c.shape[0],) + tuple(x + y - 1 for x, y in zip(sa, sb)))
52          for idx in np.ndindex(*sa):
53              coef = a.c[(slice(None),) + idx]
54              if not np.any(coef): continue
55              C[(slice(None),) + tuple(slice(i, i + n) for i, n in zip(idx, sb))] += coef.reshape((-1,) +
    (1,) * len(sb)) * b.c
56          return QN(C)
57      __rmul__ = __mul__
58      def __pow__(s, n):
59          out = QN(np.ones((s.c.shape[0],) + (1,) * (s.c.ndim - 1)))
60          for _ in range(int(n)): out = out * s
61          return out
62      def abs(s): return QN(np.abs(s.c))
63
64
65  class PieceND:
66      def __init__(s, t0, h, xs, V):
67          s.t0, s.h = t0, h; s.xs = [np.asarray(x, float) for x in xs]; s.V = [np.asarray(v, float) for v
    in V]
68
69
70  # ---------------------------------------------------------------- tensor-product spline and cell geom
    etry
71  def tensor_spline(xs, Y):
72      """Y: (N_1, ..., N_d, K) nodal data -> (cells, 4, ..., 4, K) coefficients in cell-local xi powers (C
     order of cells),
73      the tensor product of clamped cubic splines with end slope 0 on every axis"""
74      d = len(xs); A = np.asarray(Y, float)
75      for i, x in enumerate(xs):
76          pos = 2 * i; A = np.moveaxis(A, pos, 0); rest = A.shape[1:]; Y2 = A.reshape(len(x), -1); M = Y2.
    shape[1]
77          sl = spline_slopes(x, Y2, np.zeros(M), np.zeros(M)); C = cubic_coeffs(x, Y2, sl)          # (n_i
    , 4, M)
78          A = C.reshape((len(x) - 1, 4) + rest); A = np.moveaxis(A, (0, 1), (pos, pos + 1))
79      n = [len(x) - 1 for x in xs]
80      A = A.transpose([2 * i for i in range(d)] + [2 * i + 1 for i in range(d)] + [2 * d])
81      return A.reshape((int(np.prod(n)),) + (4,) * d + (A.shape[-1],))
82
83
84  def cell_geometry(xs):
85      ells = [np.diff(x) for x in xs]; grids = np.meshgrid(*ells, indexing="ij")
86      return [g.ravel() for g in grids], np.prod(np.stack([g.ravel() for g in grids]), 0)
87
88
89  def bern_nd(C):
90      """monomial -> Bernstein coefficients along every axis but the first (cells)"""
91      B = C
92      for ax in range(1, C.ndim):
93          M = bern_matrix(C.shape[ax] - 1); B = np.moveaxis(np.tensordot(M, B, axes=([1], [ax])), 0, ax)
94      return B
95
96
97  def reparam(a, b, p=4):
98      """T (len(a), p, p): coefficients of q(eta) = c(a + (b - a) eta) from c (powers of xi), new_m = sum_
    k c_k C(k,m) a^(k-m) (b-a)^m"""
99      a = np.asarray(a, float); w = np.asarray(b, float) - a; T = np.zeros((len(a), p, p))
100     for k in range(p):
101         for m in range(k + 1): T[:, m, k] = comb(k, m) * a ** (k - m) * w ** m
102     return T
103
104
105 def _apply_axis(C, T, ax, grid_shape):
106     """C: (cells as grid_shape..., coefficient axes...); T: (n_ax, p, p) indexed by the cell index along
     grid axis ax;
107     contract coefficient axis (d + ax) with T"""
108     d = len(grid_shape); G = C.reshape(tuple(grid_shape) + C.shape[1:])
109     G = np.moveaxis(G, (ax, d + ax), (0, 1))                       # (n_ax, p, ...)
110     G = np.einsum("nmk,nk...->nm...", T, G)
```

```
    G = np.moveaxis(G, (0, 1), (ax, d + ax))
    return G.reshape((C.shape[0],) + G.shape[d:])


def l1_diff_nd(xsA, cA, xsB, cB):
    """upper bound of int |pA - pB| over the box for two piecewise tensor cubics (spatial coefficients (cells, 4..4)) on
    tensor meshes xsA, xsB: union-of-knots cells per axis, both re-expanded on every union cell, Bernstein mean x volume.
    Re-expansion rounding and the rounding of the union-cell end points are covered by a pad 1e-12 sum |c| per coefficient."""
    d = len(xsA); TA, TB, iA, iB, nu, ellu = [], [], [], [], [], []
    for i in range(d):
        xu = np.union1d(xsA[i], xsB[i]); a, b = xu[:-1], xu[1:]; nu.append(len(a)); ellu.append(b - a)
        for X, T_, I_ in ((xsA[i], TA, iA), (xsB[i], TB, iB)):
            j = np.clip(np.searchsorted(X, 0.5 * (a + b)) - 1, 0, len(X) - 2); ell = X[j + 1] - X[j]
            T_.append(reparam((a - X[j]) / ell, (b - X[j]) / ell)); I_.append(j)
    out = []
    for xs, c, T_, I_ in ((xsA, cA, TA, iA), (xsB, cB, TB, iB)):
        n = [len(x) - 1 for x in xs]; J = np.meshgrid(*I_, indexing="ij")
        G = c[np.ravel_multi_index([j.ravel() for j in J], n)]
        for ax in range(d): G = _apply_axis(G, T_[ax], ax, nu)
        out.append(G)
    D = out[0] - out[1]; Bz = bern_nd(D)
    pad = 1e-12 * (np.abs(out[0]).reshape(len(D), -1).sum(1) + np.abs(out[1]).reshape(len(D), -1).sum(1))
    vol = np.prod(np.stack([g.ravel() for g in np.meshgrid(*ellu, indexing="ij")]), 0)
    return float(np.sum(vol * (np.abs(Bz).reshape(len(D), -1).mean(1) + pad))) * PADF


def eval_tensor(xs, C, xs_new):
    """values of the piecewise tensor cubic (spatial coefficients (cells, 4..4)) at the tensor grid xs_new"""
    d = len(xs); n = [len(x) - 1 for x in xs]; G = C.reshape(tuple(n) + (4,) * d)
    for i in range(d):
        x, xn = xs[i], xs_new[i]; j = np.clip(np.searchsorted(x, xn, side="right") - 1, 0, len(x) - 2)
        xi = (xn - x[j]) / (x[j + 1] - x[j]); Pw = xi[:, None] ** np.arange(4)[None, :]
        G = np.take(G, j, axis=i)                                   # cell axis i -> new points
        G = np.moveaxis(G, (i, d), (0, 1)); G = np.einsum("pk,pk...->p...", Pw, G); G = np.moveaxis(G, 0, i)
    return G


# ------------------------------------------------------------------ the judge
class JudgeND:
    def __init__(s, spec):
        """spec = dict(m, D, R, T, box=(lo, hi), dim)"""
        s.spec = spec; s.m = spec["m"]; s.D = list(spec["D"]); s.d = spec["dim"]
        s.j1 = _Judge1(dict(m=s.m, D=s.D, R=spec["R"], T=spec["T"], box=spec["box"]))   # mu_1 and the polynomial R, Rabs
        s.R, s.Rabs = s.j1.R, s.j1.Rabs; s.lo, s.hi = s.j1.lo, s.j1.hi

    def _dt(s, A):
        c = A.c; p = c.shape[-1]; out = np.zeros_like(c)
        if p > 1: out[..., :-1] = c[..., 1:] * np.arange(1, p)
        return QN(out)

    def _dii(s, A, i, ell):
        c = np.moveaxis(A.c, 1 + i, 0); p = c.shape[0]; out = np.zeros_like(c)
        if p > 2: out[:-2] = c[2:] * (np.arange(2, p) * np.arange(1, p - 1)).reshape((-1,) + (1,) * (c.ndim - 1))
        out = np.moveaxis(out, 0, 1 + i)
        return QN(out / (ell ** 2).reshape((-1,) + (1,) * (out.ndim - 1)))

    def _l1cells(s, A, tlen, vol, h):
        Rs = A.c * tlen[:A.c.shape[-1]]; B = bern_nd(Rs); n = len(Rs)
        return vol * h * (np.abs(B).reshape(n, -1).mean(1) + 1e-13 * np.abs(Rs).reshape(n, -1).sum(1))

    def _initial(s, xs, U, u0box, nsub, u0grad=None):
        """sum_a ||u~_a(0) - u0_a||_1: every cell cut into nsub^d sub-boxes; Bernstein range of the re-expanded tensor cubic
        against the user's enclosure of the exact initial data on each sub-box"""
        d = s.d; ells, vol = cell_geometry(xs); n = [len(x) - 1 for x in xs]
        sub = np.arange(nsub) / nsub; T1 = reparam(sub, sub + 1.0 / nsub)            # (nsub, 4, 4), same for every cell
        lows = [g.ravel() for g in np.meshgrid(*[x[:-1] for x in xs], indexing="ij")]
        e0 = 0.0
        for a in range(s.m):
            C0 = U[a].c[..., 0]                                                     # (cells, 4..4)
            for c0 in range(0, len(C0), 256):
```

```
                Cc = C0[c0:c0 + 256]; nc = len(Cc); G = Cc
                for ax in range(d):        # the next unexpanded coefficient axis is always axis 1; (nsub, 4) is appended at the end
                    G = np.einsum("smk,...k->...sm", T1, np.moveaxis(G, 1, -1))
                # now (nc, s_1, m_1, ..., s_d, m_d)
                G = G.reshape((nc,) + tuple(v for _ in range(d) for v in (nsub, 4)))
                G = G.transpose([0] + [1 + 2 * i for i in range(d)] + [2 + 2 * i for i in range(d)])
                G = G.reshape((nc * nsub ** d,) + (4,) * d)
                Bz = bern_nd(G); flat = Bz.reshape(len(G), -1)
                tl = 1e-13 * np.abs(G).reshape(len(G), -1).sum(1)
                clo, chi = flat.min(1) - tl, flat.max(1) + tl
                # sub-box corners
                idx = np.array(np.meshgrid(*[np.arange(nsub)] * d, indexing="ij")).reshape(d, -1)  # (d, nsub^d)
                xl, xh = [], []
                for i in range(d):
                    lo_c = lows[i][c0:c0 + nc][:, None]; el = ells[i][c0:c0 + nc][:, None]
                    xl.append((lo_c + el * idx[i][None, :] / nsub).ravel()); xh.append((lo_c + el * (idx[i][None, :] + 1) / nsub).ravel())
                xl = [v - 4e-16 * np.abs(v) - 1e-300 for v in xl]; xh = [v + 4e-16 * np.abs(v) + 1e-300 for v in xh]   # cover the rounding of the corners
                sv = np.repeat(vol[c0:c0 + nc] / nsub ** d, nsub ** d)
                if u0grad is None:             # zeroth order: |e| <= max(|u0_hi - c_lo|, |c_hi - u0_lo|) on the sub-box
                    ulo, uhi = u0box(a, xl, xh)
                    e0 += float(np.sum(np.maximum(np.abs(uhi - clo), np.abs(chi - ulo)) * sv))
                else:                          # mean-value form about the sub-box centre z:  |e(x)| <= |e(z)| + sum_i max|d_i e| r_i
                    wv = 0.5 ** np.arange(4); Gc = G
                    for _ in range(d): Gc = np.tensordot(Gc, wv, axes=([1], [0]))
                    tlc = 1e-13 * np.abs(G).reshape(len(G), -1).sum(1)
                    zc = [0.5 * (l + h_) for l, h_ in zip(xl, xh)]; zlo, zhi = u0box(a, zc, zc)
                    bound = np.maximum(np.abs(zhi - (Gc - tlc)), np.abs((Gc + tlc) - zlo))
                    glo_hi = u0grad(a, xl, xh)
                    for i in range(d):
                        Gd = np.moveaxis(G, 1 + i, 0); Dd = np.zeros_like(Gd); Dd[:-1] = Gd[1:] * np.arange(1, 4).reshape((-1,) + (1,) * (Gd.ndim - 1))
                        Dd = np.moveaxis(Dd, 0, 1 + i); sc = np.repeat(nsub / ells[i][c0:c0 + nc], nsub ** d)
                        Bd = bern_nd(Dd).reshape(len(G), -1); tld = 1e-13 * np.abs(Dd).reshape(len(G), -1).sum(1)
                        dlo, dhi = (Bd.min(1) - tld) * sc, (Bd.max(1) + tld) * sc
                        glo, ghi = glo_hi[i]; r = 0.5 * (xh[i] - xl[i])
                        bound = bound + np.maximum(np.abs(ghi - dlo), np.abs(dhi - glo)) * r
                    e0 += float(np.sum(bound * sv)) * (1 + 1e-12)
        return e0 * PADF

    def run(s, pieces, u0box, nsub=4, op=None, u0grad=None):
        """u0box(a, xlo_list, xhi_list) -> (lo, hi) arrays: rigorous enclosure of the exact initial data on the sub-boxes.
        u0grad(a, xlo_list, xhi_list) -> [(lo_i, hi_i)] (optional): enclosures of the partial derivatives of the initial data;
        when given, the initial error uses the mean-value form (second order in the sub-box size).
        op(xs) -> (L, w, src) (optional): the spatial operator on the tensor mesh xs (flattened in C order); when given,
        every cell residual is also split into its time-scheme part and its spatial part (localize)."""
        m, d = s.m, s.d; Eb = 0.0; e0 = None; prev = None; LOC = []; series = []; tw = time.time()
        resid = rnd_tot = jumps = 0.0; mu_max = -np.inf
        for Pc in pieces:
            xs, h = Pc.xs, Pc.h; K = Pc.V[0].shape[-1]; ells, vol = cell_geometry(xs); tlen = h ** np.arange(4 * K + 4)
            U = [QN(tensor_spline(xs, Pc.V[a])) for a in range(m)]
            if e0 is None: e0 = s._initial(xs, U, u0box, nsub, u0grad); Eb = e0
            Rv = s.R(*U); Ra = s.Rabs(*[u.abs() for u in U])
            if op is not None:
                Lop, _, src = op(xs); Nn = int(np.prod([len(x) for x in xs]))
                Pn = [_P(Pc.V[b].reshape(Nn, K).T) for b in range(m)]; Rn = [_P.k(v, Nn) for v in s.R(*Pn)]
                srcm = np.asarray(src, float).reshape(m, -1) if np.size(src) > 1 else np.zeros((m, Nn))
            tot = 0.0; rnd_p = 0.0; cell = []; frac = []
            for a in range(m):
                lap = s._dii(U[a], 0, ells[0])
                for i in range(1, d): lap = lap + s._dii(U[a], i, ells[i])
                ra = s._dt(U[a]) - s.D[a] * lap - (Rv[a] if isinstance(Rv[a], QN) else QN.k(Rv[a], U[a]))
                cv = s._l1cells(ra, tlen, vol, h); cell.append(cv); tot += float(cv.sum())
                mag = s._dt(U[a]).abs() + abs(s.D[a]) * QN(sum(np.abs(s._dii(U[a], i, ells[i]).c) for i in range(d)))
                mag = mag + (Ra[a] if isinstance(Ra[a], QN) else QN.k(abs(Ra[a]), U[a]))
                nops = 4 * int(np.prod(mag.c.shape[1:])) * (1 + d)
```

```
                rr = gam(nops) * 2 * float(np.sum(vol * h * np.sum((mag.c * tlen[:mag.c.shape[-1]]).resh
ape(len(vol), -1), 1)))
                tot += rr; rnd_p += rr
                if op is not None:
                    Va = Pc.V[a].reshape(Nn, K)
                    dl = Pn[a].der() - _P((Lop @ Va).T * s.D[a]) - _P.k(srcm[a], Nn) - Rn[a]
                    Kd = dl.c.shape[0]; rt = QN(tensor_spline(xs, dl.c.T.reshape(tuple(len(x) for x in x
s) + (Kd,))))
                    at = s._l1cells(rt, tlen, vol, h); ax_ = s._l1cells(ra - rt, tlen, vol, h)
                    frac.append(np.where(at + ax_ > 0, at / np.maximum(at + ax_, 1e-300), 0.5))
                else:
                    frac.append(None)
            resid += tot - rnd_p; rnd_tot += rnd_p
            # candidate range on this piece -> L1 log-norm of the reaction Jacobian on the hull with the
 invariant box
            lo, hi = s.lo.copy(), s.hi.copy()
            for a in range(m):
                Us = U[a].c * tlen[:K]; Bz = bern_nd(Us).reshape(len(Us), -1); tl = 1e-13 * np.abs(Us).r
eshape(len(Us), -1).sum(1)
                lo[a] = min(lo[a], float((Bz.min(1) - tl).min())); hi[a] = max(hi[a], float((Bz.max(1) +
 tl).max()))
            mu = s.j1.mu1(lo, hi); mu_max = max(mu_max, mu)
            starts = [u.c[..., 0] for u in U]
            if prev is not None:
                xp, ends, rads = prev
                jl = [l1_diff_nd(xp, ends[a], xs, starts[a]) + rads[a] for a in range(m)]; jumps += sum(
jl)
            else:
                jl = [0.0] * m
            g = math.exp(max(mu, 0.0) * h) * (1 + 1e-15)
            LOC.append(dict(t0=Pc.t0, h=h, xs=xs, cell=cell, dom=frac, jump=jl, rounding=rnd_p, g=g))
            Eb = g * (Eb + sum(jl) + tot)
            ends = [np.tensordot(u.c, tlen[:u.c.shape[-1]], axes=([-1], [0])) for u in U]
            rads = [gam(2 * K + 4) * float(np.sum(vol * np.sum(np.abs(u.c * tlen[:K]).reshape(len(vol),
-1), 1))) for u in U]
            prev = (xs, ends, rads); series.append([Pc.t0 + h, Eb])
        W = 1.0
        for L_ in reversed(LOC): W *= L_["g"]; L_["W"] = W
        return dict(loc=LOC, pieces=len(LOC), e0=e0, residual=resid, rounding=rnd_tot, jumps=jumps, boun
d=Eb * PADF,
                    mu1_max=mu_max, series=series, wall=time.time() - tw, dim=d)


def localize(res, top=10):
    """exact additive split of the bound (see pde_judge.localize); cells are boxes, reported by their co
rner ranges"""
    B = res["bound"] / PADF; L = res["loc"]; rows = []
    tot = dict(initial=L[0]["W"] * res["e0"], time_scheme=0.0, space=0.0, residual=0.0, jump=0.0, roundi
ng=0.0)
    for P_ in L:
        W = P_["W"]; xs = P_["xs"]; n = [len(x) - 1 for x in xs]
        for a, (cv, ft) in enumerate(zip(P_["cell"], P_["dom"])):
            parts = [("residual", cv)] if ft is None else [("time_scheme", cv * ft), ("space", cv * (1 -
 ft))]
            for name, arr in parts:
                for j in np.argsort(-arr)[:top]:
                    ij = np.unravel_index(j, n)
                    rows.append(dict(t=[P_["t0"], P_["t0"] + P_["h"]], x=[[float(xs[i][ij[i]]), float(xs
[i][ij[i] + 1])] for i in range(len(xs))],
                                     species=a, cause=name, share=W * float(arr[j]) / B))
                tot[name] += W * float(np.sum(arr))
        for a, jv in enumerate(P_["jump"]):
            if jv > 0: rows.append(dict(t=[P_["t0"], P_["t0"]], x=None, species=a, cause="jump", share=W
 * jv / B))
            tot["jump"] += W * jv
        tot["rounding"] += W * P_["rounding"]
    rows.sort(key=lambda r: -r["share"])
    return rows[:top], {k: v / B for k, v in tot.items() if v > 0 or k in ("initial", "jump")}
```

orders_nd.py — 텐서 격자 위의 공간 먼저·시간 먼저(축별 재배치) 후보

```
"""orders_nd: candidates on tensor meshes in any dimension d for pde_judge_nd (zero-flux box [0, Lx]^d).
  kron_operator(xs)        P1 lumped-mass operator on the tensor mesh: L = sum_i I x .. x L_i x .. x I (
sparse), w = w_1 x .. x w_d
  mol_nd(prob, xs, n)      SPACE FIRST: RK4 on the fixed mesh, joined in time by cubic Hermite interpola
tion
  rothe_nd(prob, xs, n, remesh)  TIME FIRST: every step of the 2-stage Radau IIA scheme (order 3, L-stab
le) is an elliptic
                           problem solved on that step's mesh; every `remesh` steps each axis is re-equi
```

```
    distributed (arc-length
 6                            monitor averaged over the other axes) and the solution is carried over by eva
    luating the tensor spline
 7    pieces_from_steps(prob, steps)  ANY time scheme: steps = [(t0, h, xs, y0, y1)], the values at both end
    s of each step on
 8                            that step's mesh (variable steps and changing meshes allowed), joined by cubi
    c Hermite with slopes F
 9  prob = dict(m, D, R, T, u0 (list of functions of the coordinate list)).  All return a list of PieceND.""
    "
10  import numpy as np
11  import sympy as sp
12  import scipy.sparse as sps
13  from scipy.sparse.linalg import splu
14  from operators import fem_p1
15  from pde_judge_nd import PieceND, tensor_spline, eval_tensor
16
17
18  def kron_operator(xs):
19      ops = [fem_p1(x) for x in xs]; Ls = [sps.csr_matrix(o[0]) for o in ops]; ws = [o[1] for o in ops]; n
     = [len(x) for x in xs]
20      L = None
21      for i in range(len(xs)):
22          f = [sps.identity(k, format="csr") for k in n]; f[i] = Ls[i]; T = f[0]
23          for g in f[1:]: T = sps.kron(T, g, format="csr")
24          L = T if L is None else L + T
25      w = ws[0]
26      for v in ws[1:]: w = np.kron(w, v)
27      return L.tocsr(), w, np.zeros(L.shape[0])
28
29
30  class _RD:
31      def __init__(s, prob):
32          s.m = prob["m"]; s.D = prob["D"]; sy = sp.symbols(f"s0:{s.m}"); Rs = [sp.sympify(e) for e in pro
    b["R"](list(sy))]
33          s.Rn = sp.lambdify(sy, Rs, "numpy"); s.Jn = [[sp.lambdify(sy, sp.diff(Rs[a], sy[b]), "numpy") fo
    r b in range(s.m)] for a in range(s.m)]
34
35      def F(s, z, L):
36          N = L.shape[0]; u = z.reshape(s.m, N); r = s.Rn(*u)
37          return np.concatenate([s.D[a] * (L @ u[a]) + np.asarray(r[a], float) * np.ones(N) for a in range
    (s.m)])
38
39      def J(s, z, L):
40          N = L.shape[0]; u = z.reshape(s.m, N)
41          return sps.bmat([[sps.diags(np.asarray(s.Jn[a][b](*u), float) * np.ones(N)) + (s.D[a] * L if a =
    = b else None)
42                           for b in range(s.m)] for a in range(s.m)], format="csc")
43
44
45  def _init(prob, xs):
46      X = np.meshgrid(*xs, indexing="ij"); return np.concatenate([f(X).ravel() for f in prob["u0"]])
47
48
49  def _piece(t, h, xs, cs, m):
50      shp = tuple(len(x) for x in xs); N = int(np.prod(shp))
51      return PieceND(t, h, xs, [cs[:, a * N:(a + 1) * N].T.reshape(shp + (cs.shape[0],)) for a in range(m)
    ])
52
53
54  def mol_nd(prob, xs, n):
55      rd = _RD(prob); L, _, _ = kron_operator(xs); y = _init(prob, xs); h = prob["T"] / n; out = []
56      for k in range(n):
57          k1 = rd.F(y, L); k2 = rd.F(y + h / 2 * k1, L); k3 = rd.F(y + h / 2 * k2, L); k4 = rd.F(y + h * k
    3, L)
58          b = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4); fb = rd.F(b, L); dl = b - y
59          cs = np.array([y, k1, (3 * dl / h - 2 * k1 - fb) / h, (k1 + fb - 2 * dl / h) / h ** 2])     # cu
    bic Hermite in s
60          out.append(_piece(k * h, h, xs, cs, rd.m)); y = b
61      return out
62
63
64  def _equi(x, prof, alpha=4.0):
65      xm = 0.5 * (x[1:] + x[:-1]); dx = np.diff(x); om = np.sqrt(1 + alpha ** 2 * (np.diff(prof) / dx) **
    2)
66      kk = min(5, len(om)); om = np.convolve(om, np.ones(kk) / kk, mode="same"); cum = np.concatenate([[0]
    , np.cumsum(om * dx)]); cum /= cum[-1]
67      xn = np.interp(np.linspace(0, 1, len(x)), cum, x); xn[0], xn[-1] = x[0], x[-1]; return xn
68
69
70  def rothe_nd(prob, xs, n, remesh=10):
```

```
    rd = _RD(prob); m = rd.m; A = np.array([[5 / 12, -1 / 12], [3 / 4, 1 / 4]]); h = prob["T"] / n
    Lag = np.linalg.inv(np.vander(np.array([0.0, 1 / 3, 1.0]), 3, increasing=True))
    xs = [x.copy() for x in xs]; L, _, _ = kron_operator(xs); y = _init(prob, xs); out = []; meshes = [(0.0, xs)]
    for k in range(n):
        if k > 0 and k % remesh == 0:          # per-axis re-equidistribution, transfer by the tensor spline
            shp = tuple(len(x) for x in xs); N = int(np.prod(shp)); ys = [y[a * N:(a + 1) * N].reshape(shp) for a in range(m)]
            new = []
            for i, x in enumerate(xs):
                prof = np.mean(np.moveaxis(ys[0], i, 0).reshape(len(x), -1), 1); new.append(_equi(x, prof))
            y = np.concatenate([eval_tensor(xs, tensor_spline(xs, ys[a][..., None])[..., 0], new).ravel() for a in range(m)])
            xs = new; L, _, _ = kron_operator(xs); meshes.append((k * h, xs))
        M = len(y); Y = np.concatenate([y, y])
        for _ in range(30):
            F1, F2 = rd.F(Y[:M], L), rd.F(Y[M:], L); J1, J2 = rd.J(Y[:M], L), rd.J(Y[M:], L)
            G = Y - np.concatenate([y, y]) - h * np.concatenate([A[0, 0] * F1 + A[0, 1] * F2, A[1, 0] * F1 + A[1, 1] * F2])
            Jb = sps.identity(2 * M, format="csc") - h * sps.bmat([[A[0, 0] * J1, A[0, 1] * J2], [A[1, 0] * J1, A[1, 1] * J2]], format="csc")
            d = splu(Jb).solve(G); Y -= d
            if np.max(np.abs(d)) < 1e-14 * max(1.0, np.max(np.abs(Y))): break
        cs = (Lag @ np.stack([y, Y[:M], Y[M:]])) / (h ** np.arange(3))[:, None]
        out.append(_piece(k * h, h, xs, cs, m)); y = Y[M:].copy()
    return out, meshes


def pieces_from_steps(prob, steps):
    rd = _RD(prob); out = []; cache = {}
    for t0, h, xs, y0, y1 in steps:
        key = tuple(len(x) for x in xs) + tuple(float(x.sum()) for x in xs)
        if key not in cache: cache[key] = kron_operator(xs)[0]
        L = cache[key]; f0, f1 = rd.F(y0, L), rd.F(y1, L); dl = y1 - y0
        cs = np.array([y0, f0, (3 * dl / h - 2 * f0 - f1) / h, (f0 + f1 - 2 * dl / h) / h ** 2])
        out.append(_piece(t0, h, xs, cs, rd.m))
    return out
```

integrator.py — 테일러 적분기(식 그래프, 자동 라이프니츠 점화, 적응 걸음)

```
"""integrator: the time integrator whose polynomials serve as Taylor candidates and reference solutions.

Weakness 1 (hand-derived recurrences)
  QES-J  : the right-hand side f is written ONCE as an expression graph (ordinary operators).
           Relaxed Taylor-mode evaluation applies the Leibniz rule automatically, coefficient by
           coefficient. An algebraic nonlinearity b = B(a) given only by a polynomial relation
           g(a, b) = 0 is solved by Newton iteration at order 0 and by one linearised Newton step
           at every higher order (the step is exact because the order-j equation is linear in b_j).
  QES-G8 : implicit 4-stage Gauss-Legendre (order 8; stability function = (4,4) Pade of e^z, the
           book's QES-P with m = 4). Stage equations by Newton-Krylov using only f.

Weakness 2 (problem-specific step analysis)
  2A : generic stability step h_S = r8 / R. R = spectral radius of the Jacobian estimated by power
       iteration with black-box products J v = (f(y + eps v) - f(y)) / eps. r8 = 3.395 is the radius
       of the largest left half-disk inside {|S_8(z)| <= 1}: if every eigenvalue lies in the left
       half-disk of radius R, then h R <= r8 puts h*lambda inside the stability region.
  2B : Cauchy majorant |c_j| <= M rho^-j fitted to ALL computed coefficients (M = |c_0|,
       rho = min_j (M/|c_j|)^(1/j)); accuracy step theta*rho with theta^9/(1-theta) <= eps_k and
       per-step tail estimate M theta^9/(1-theta) (Cantor tail).
"""
import math
import numpy as np
from scipy.sparse.linalg import LinearOperator, gmres

R8 = 3.3955


def eps_ladder(k):
    return 1.0 / (math.factorial(k) * k)


def theta_from_ladder(n, eps):
    lo, hi = 0.0, 0.999
    for _ in range(80):
        m = 0.5 * (lo + hi)
        lo, hi = (m, hi) if m ** (n + 1) / (1 - m) <= eps else (lo, m)
    return lo
```

```

# ============================================================================ expression graph
class Node:
    _ops = 0                      # counts linear-operator applications (FFT pairs)

    def __init__(self, kind, kids=(), fn=None, val=None):
        self.kind, self.kids, self.fn, self.val = kind, list(kids), fn, val
        self.c = []

    def __add__(s, o): return Node("add", [s, _n(o)])
    def __radd__(s, o): return Node("add", [_n(o), s])
    def __sub__(s, o): return Node("sub", [s, _n(o)])
    def __rsub__(s, o): return Node("sub", [_n(o), s])
    def __mul__(s, o):
        if np.isscalar(o):
            return Node("scale", [s], val=float(o))
        return Node("mul", [s, _n(o)])
    def __rmul__(s, o): return s.__mul__(o)
    def __neg__(s): return Node("scale", [s], val=-1.0)


def _n(o):
    return o if isinstance(o, Node) else Node("const", val=o)


def lin(fn, a):
    """linear operator (spectral derivative, Poisson solve, filter ...): applied per coefficient."""
    return Node("lin", [a], fn=fn)


def mlin(fn, a):
    """several linear operators of the same input in one call (stacked FFT)."""
    return Node("mlin", [a], fn=fn)


def pick(m, i):
    return Node("pick", [m], val=i)


def algebraic(g, a, b0_guess):
    """b defined by the polynomial relation g(a, b) = 0 (elementwise). g must use only + - * and
    scalars, so it runs on numbers, arrays and truncated series alike."""
    return Node("alg", [a], fn=g, val=b0_guess)


class Var(Node):
    def __init__(self):
        super().__init__("var")


# ------------------------------------------------------------------ truncated series for g(a, b)
class Ser:
    def __init__(self, c):
        self.c = list(c)
    def _o(s, o):
        return o if isinstance(o, Ser) else Ser([o] + [0.0] * (len(s.c) - 1))
    def __add__(s, o): o = s._o(o); return Ser([x + y for x, y in zip(s.c, o.c)])
    __radd__ = __add__
    def __sub__(s, o): o = s._o(o); return Ser([x - y for x, y in zip(s.c, o.c)])
    def __rsub__(s, o): o = s._o(o); return Ser([y - x for x, y in zip(s.c, o.c)])
    def __mul__(s, o):
        if np.isscalar(o):
            return Ser([o * x for x in s.c])
        n = len(s.c)
        return Ser([sum(s.c[i] * o.c[j - i] for i in range(j + 1)) for j in range(n)])
    __rmul__ = __mul__


def topo(out):
    seen, order = set(), []
    def visit(v):
        if id(v) in seen:
            return
        seen.add(id(v))
        for k in v.kids:
            visit(k)
        order.append(v)
    visit(out)
    return order
```

```


class Graph:
    """y' = f(y) with f given as a graph from Var x to node out."""

    def __init__(self, build):
        self.x = Var()
        self.out = build(self.x)
        self.order = topo(self.out)
        self.lin_calls = 0
        self.newton_iters = []

    # ---- coefficient j of every node (relaxed Taylor mode)
    def _coef(self, v, j):
        k = v.kids
        if v.kind == "var":
            return v.c[j]                                   # set by the driver
        if v.kind == "const":
            return v.val if j == 0 else 0.0 * v.val if not np.isscalar(v.val) else 0.0
        if v.kind == "add":
            return k[0].c[j] + k[1].c[j]
        if v.kind == "sub":
            return k[0].c[j] - k[1].c[j]
        if v.kind == "scale":
            return v.val * k[0].c[j]
        if v.kind == "mul":
            return sum(k[0].c[i] * k[1].c[j - i] for i in range(j + 1))
        if v.kind == "lin":
            self.lin_calls += 1
            return v.fn(k[0].c[j])
        if v.kind == "mlin":
            self.lin_calls += 1
            return v.fn(k[0].c[j])
        if v.kind == "pick":
            return k[0].c[j][v.val]
        if v.kind == "alg":
            a = k[0].c
            g = v.fn
            if j == 0:                                      # Newton iteration at order 0
                b = np.broadcast_to(v.val, np.shape(a[0])).astype(float).copy() if np.ndim(v.val) == 0 e
lse v.val.copy()
                for it in range(1, 40):
                    s = g(Ser([a[0], 0.0 * a[0]]), Ser([b, 1.0 + 0.0 * b]))   # (g, g_b) by a dual numbe
r
                    db = s.c[0] / s.c[1]
                    b = b - db
                    if np.max(np.abs(db)) <= 1e-15 * max(1.0, np.max(np.abs(b))):
                        break
                self.newton_iters.append(it)
                v.gb = g(Ser([a[0], 0.0 * a[0]]), Ser([b, 1.0 + 0.0 * b])).c[1]
                return b
            # order j: one linearised Newton step (exact): g_b * b_j + [g(a, b with b_j = 0)]_j = 0
            bt = v.c[:j] + [0.0 * v.c[0]]
            r = g(Ser(a[:j + 1]), Ser(bt)).c[j]
            return -r / v.gb
        raise ValueError(v.kind)

    def taylor(self, y, n):
        """coefficients c_0..c_n of y(t+s); returns list of arrays."""
        for v in self.order:
            v.c = []
        self.x.c = [y]
        for j in range(n + 1):
            for v in self.order:
                if v is self.x:
                    continue
                v.c.append(self._coef(v, j))
            if j < n:
                self.x.c.append(self.out.c[j] / (j + 1))
        return self.x.c

    def f(self, y):
        """plain evaluation (coefficient 0 only) = black-box right-hand side."""
        for v in self.order:
            v.c = []
        self.x.c = [y]
        for v in self.order:
            if v is not self.x:
                v.c.append(self._coef(v, 0))
        return self.out.c[0]
```

```
196
197
198 # ============================================================================ generic steps
199 def majorant(c):
200     """|c_j| <= M rho^-j for all computed j (2B)."""
201     m = [float(np.max(np.abs(x))) for x in c]
202     M = m[0]
203     rho = min((M / mj) ** (1.0 / j) for j, mj in enumerate(m) if j > 0 and mj > 0)
204     return M, rho
205
206
207 def spectral_radius(G, y, v0=None, iters=12):
208     """power iteration with black-box J v (2A). returns (R, last vector, f-evaluations)."""
209     f0 = G.f(y); ev = 1
210     rng = np.random.default_rng(1)
211     v = rng.standard_normal(y.shape) if v0 is None else v0
212     v /= np.linalg.norm(v)
213     R = 0.0
214     for _ in range(iters):
215         eps = 1e-7 * max(1.0, np.linalg.norm(y))
216         w = (G.f(y + eps * v) - f0) / eps; ev += 1
217         R = np.linalg.norm(w); v = w / R
218     return R, v, ev
219
220
221 def qes_j(G, y0, T, n=8, k_ladder=13, cap="generic", h_spec=None, record=False, R_every=4, R_safety=1.1)
    :
222     """QES-J adaptive: h = min(theta*rho_majorant, h_S). cap in {'generic','specific', None}."""
223     theta = theta_from_ladder(n, eps_ladder(k_ladder))
224     y, t, hist, est, v, R, extra, step = y0.copy(), 0.0, [], 0.0, None, None, 0, 0
225     while t < T - 1e-14:
226         c = G.taylor(y, n)
227         M, rho = majorant(c)
228         h_acc = theta * rho
229         if cap == "generic":
230             if step % R_every == 0:
231                 R, v, e = spectral_radius(G, y, v, iters=12 if step == 0 else 4); extra += e
232             h_S = R8 / (R_safety * R)
233         elif cap == "specific":
234             h_S = h_spec(y)
235         else:
236             h_S = np.inf
237         h = min(h_acc, h_S, T - t)
238         tail = M * (h / rho) ** (n + 1) / (1 - h / rho)
239         est += tail
240         if record:
241             hist.append(dict(t=t, h=h, h_acc=h_acc, h_S=h_S, rho=rho, tail=tail))
242         s = c[-1]
243         for cj in reversed(c[:-1]):
244             s = cj + h * s
245         y = s; t += h; step += 1
246     return y, dict(steps=step, hist=hist, tail_est=est, extra_f=extra)
247
248
249 def qes_j_fixed(G, y0, T, steps, n=8):
250     y, h = y0.copy(), T / steps
251     for _ in range(steps):
252         c = G.taylor(y, n)
253         s = c[-1]
254         for cj in reversed(c[:-1]):
255             s = cj + h * s
256         y = s
257     return y
258
259
260 # ============================================================================ QES-G8 (implicit)
261 def gauss_legendre(s=4):
262     x, w = np.polynomial.legendre.leggauss(s)
263     c = (x + 1) / 2; b = w / 2
264     A = np.zeros((s, s))
265     for j in range(s):
266         others = [c[m] for m in range(s) if m != j]
267         p = np.poly1d(np.poly(others)) / np.prod([c[j] - o for o in others])
268         P = np.polyint(p)
269         A[:, j] = P(c) - P(0)
270     return A, b, c
271
272
273 GL_A, GL_B, GL_C = gauss_legendre(4)
274 GL_LAM, GL_V = np.linalg.eig(GL_A)
```

```
GL_Vi = np.linalg.inv(GL_V)


def qes_g8(G, y0, T, steps, lin_symbol=None, fft=None, ifft=None, newton_tol=1e-12, gm_tol=1e-8, maxit=60, record=None):
    """fixed-step implicit Gauss-Legendre (order 8). K_i = f(y + h sum_j A_ij K_j).
    Newton-Krylov: J-v by finite differences of f only. Optional preconditioner from a diagonal
    linear-part symbol L (spectral): P = I - h A (x) L, inverted mode by mode through A = V Lam V^-1."""
    h = T / steps; y = y0.copy(); s = 4; shp = y.shape; nd = y.size
    cnt = dict(f=0, gmres=0, newton=0, unconverged=0)

    def F(Y):
        cnt["f"] += 1
        return G.f(Y)

    def stages(K):
        return [y + h * sum(GL_A[i, j] * K[j] for j in range(s)) for i in range(s)]

    def prec(r):
        R = r.reshape(s, *shp)
        if lin_symbol is None:
            return r
        Rh = [fft(R[i]) for i in range(s)]
        Z = [sum(GL_Vi[m, i] * Rh[i] for i in range(s)) for m in range(s)]
        Z = [Z[m] / (1 - h * GL_LAM[m] * lin_symbol) for m in range(s)]
        X = [sum(GL_V[i, m] * Z[m] for m in range(s)) for i in range(s)]
        return np.concatenate([np.real(ifft(X[i])).ravel() for i in range(s)])

    for _ in range(steps):
        f0 = F(y)
        K = [f0.copy() for _ in range(s)]
        for it in range(12):
            Ys = stages(K); fY = [F(Yi) for Yi in Ys]
            res = np.concatenate([(K[i] - fY[i]).ravel() for i in range(s)])
            def mv(d):
                D = d.reshape(s, *shp)
                out = []
                for i in range(s):
                    dY = h * sum(GL_A[i, j] * D[j] for j in range(s))
                    nrm = np.linalg.norm(dY)
                    if nrm == 0:
                        out.append(D[i].ravel()); continue
                    eps = 1e-7 * max(1.0, np.linalg.norm(Ys[i])) / nrm
                    out.append((D[i] - (F(Ys[i] + eps * dY) - fY[i]) / eps).ravel())
                return np.concatenate(out)
            Aop = LinearOperator((s * nd, s * nd), matvec=mv)
            Mop = LinearOperator((s * nd, s * nd), matvec=prec)
            iters = [0]
            def cb(_):
                iters[0] += 1
            d, info = gmres(Aop, -res, rtol=gm_tol, atol=0.0, restart=maxit, maxiter=1, M=Mop, callback=cb, callback_type="pr_norm")
            cnt["gmres"] += iters[0]; cnt["newton"] += 1
            if info != 0:
                cnt["unconverged"] += 1
            D = d.reshape(s, *shp)
            K = [K[i] + D[i] for i in range(s)]
            if np.linalg.norm(d) <= newton_tol * max(1.0, np.linalg.norm(np.concatenate([k.ravel() for k in K]))):
                break
        y = y + h * sum(GL_B[i] * K[i] for i in range(s))
    return y, cnt
```

viz.py — 렌더링 규칙(잘림·겹침·빈 패널 검사, 증명 구간 띠)

```
"""viz: the rendering rules of the IDE (15.2).  The full time range is always drawn; the proved interval [0, t*] is a
strip above the panel; every figure passes a clipping / overlap / empty-panel check before it is saved."""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.colors import LinearSegmentedColormap

GOOD = "#0ca30c"
BLUE, ORANGE, AQUA, RED, GRAY = "#2a78d6", "#eb6834", "#1baf7a", "#d03b3b", "#c3c2b7"
SEQ = LinearSegmentedColormap.from_list("seq", ["#ffffff", "#cde2fb", "#86b6ef", "#3987e5", "#1c5cab", "
```

```
    #0d366b"])
13  try:
14      font_manager.fontManager.addfont("/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc")
15      plt.rcParams.update({"font.family": "Noto Sans CJK JP"})
16  except Exception:
17      pass
18
19
20  def check_clip(fig, name):
21      fig.canvas.draw(); r = fig.canvas.get_renderer(); W, H = fig.bbox.width, fig.bbox.height; bad = []
22      arts = [t for t in fig.findobj(matplotlib.text.Text) if t.get_visible() and t.get_text().strip()]
23      arts += [l for l in fig.findobj(matplotlib.legend.Legend)]
24      for a in arts:
25          bb = a.get_window_extent(r)
26          if bb.x0 < -0.5 or bb.y0 < -0.5 or bb.x1 > W + 0.5 or bb.y1 > H + 0.5:
27              bad.append((type(a).__name__, getattr(a, "get_text", lambda: "legend")()[:30], [round(v) for
     v in bb.extents]))
28      # overlap among titles, legends, strip labels and strip bars (tick labels excluded)
29      key = [t for t in fig.findobj(matplotlib.text.Text) if t.get_visible() and t.get_text().strip() and
    getattr(t, "_ide_key", False)]
30      key += list(fig.findobj(matplotlib.legend.Legend)) + [p for p in fig.findobj(matplotlib.patches.Rect
    angle) if getattr(p, "_ide_key", False)]
31      ext = [(a, a.get_window_extent(r)) for a in key]
32      for i in range(len(ext)):
33          for j in range(i + 1, len(ext)):
34              A, B = ext[i][1], ext[j][1]
35              if min(A.x1, B.x1) - max(A.x0, B.x0) > 0.5 and min(A.y1, B.y1) - max(A.y0, B.y0) > 0.5:
36                  if isinstance(ext[i][0], matplotlib.patches.Rectangle) and isinstance(ext[j][0], matplot
    lib.patches.Rectangle): continue
37                  bad.append(("overlap", str(ext[i][0])[:40], str(ext[j][0])[:40]))
38      for ax in fig.axes:                             # an axes without data is a cut simulation
39          if ax.get_visible() and ax.get_label() != "<colorbar>" and not (ax.lines or ax.collections or ax
    .images):
40              bad.append(("empty panel", ax.get_title()[:30]))
41      if bad: raise RuntimeError(f"{name}: clipped/overlap {bad}")
42      return True
43
44
45  def place_legend(ax, **kw):
46      """try the nine inner positions; keep the first whose box covers no plotted point, else put it below
     the axes"""
47      fig = ax.figure
48      for loc in ("upper right", "upper left", "lower right", "lower left", "center right", "center left",
     "upper center", "lower center", "center"):
49          lg = ax.legend(loc=loc, **kw); fig.canvas.draw(); bb = lg.get_window_extent(fig.canvas.get_rende
    rer())
50          hit = 0
51          for ln in ax.lines:
52              xy = ax.transData.transform(np.column_stack(ln.get_data()))
53              hit += int(np.sum((xy[:, 0] > bb.x0) & (xy[:, 0] < bb.x1) & (xy[:, 1] > bb.y0) & (xy[:, 1] <
     bb.y1)))
54          if hit == 0: return lg
55      return ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.22), ncol=3, **kw)
56
57
58  def strip(ax, T, cands):
59      """thin strip above the panel: one row per candidate, green = proved [0,t*], hatched gray = not prov
    ed"""
60      y = 1.02
61      for cn, r in cands:
62          ts = r["t_star"]
63          for pch in (plt.Rectangle((0, y), ts, 0.045, transform=ax.get_xaxis_transform(), color=GOOD, alp
    ha=0.85, clip_on=False, lw=0),
64                      plt.Rectangle((ts, y), T - ts, 0.045, transform=ax.get_xaxis_transform(), facecolor=
    "#f2f1ed",
65                                    edgecolor=GRAY, hatch="////", lw=0.3, clip_on=False)):
66              pch._ide_key = True; ax.add_patch(pch)
67          dv = r.get("dominant") == "candidate diverged" or (r.get("hist") or [{}])[0].get("dominant") ==
    "candidate diverged"
68          tx = ax.text(1.01, y + 0.0225, f"{cn}  " + ("발산" if dv else f"[0, {ts:.4g}]"), transform=ax.tr
    ansAxes, fontsize=6, va="center", ha="left", color="#52514e")
69          tx._ide_key = True
70          y += 0.065
71      return y
```